

Package ‘PopGenR’

June 4, 2026

Type Package
Title Population Genetics Statistics and Selection Scan Utilities
Version 0.1.0
Author Abu Bakar [aut, cre],
Waqar ul Haq [aut]
Maintainer Abu Bakar <malikabubakar279@gmail.com>
Description Provides utilities for population genetics workflows,
including VCF parsing, allele-frequency summaries, nucleotide
diversity, Watterson's theta, Tajima's D, pooled heterozygosity,
linkage disequilibrium, PLINK-based quality control, and gene
annotation of genomic regions or single positions.
License MIT + file LICENSE
URL <https://malikbak.github.io/PopGenR/>,
<https://github.com/malikbak/PopGenR>
BugReports <https://github.com/malikbak/PopGenR/issues>
Encoding UTF-8
LazyData true
Depends R (>= 4.0.0)
Imports data.table,
dplyr,
ggplot2,
vcfR
Suggests pkgdown,
rtracklayer,
roxygen2
Config/roxygen2/version 8.0.0

Contents

allele.freq	2
AnnoRan	3
AnnoSin	3
calc_r2	4
count.genotypes	5

derivedCount	6
dnuc	7
expected.het	8
fixed.poly	9
get.field	10
get.haplotypes	11
Hetp	12
maf	12
Mbed	13
pi.diversity	14
plQC	15
read.vcf	16
RunHom	17
TajimaD	18
variance.d	18
watsonson.theta	19

allele.freq

allele.freq: Calculate Allele Frequencies from Genotype Counts

Description

This function computes the allele frequencies for a bi-allelic locus (with alleles ‘A’ and ‘a’) based on genotype counts. It calculates the frequency of the dominant allele (‘p’) and the recessive allele (‘q’).

Usage

```
allele.freq(genotypeCounts)
```

Arguments

genotypeCounts A named numeric vector with counts for genotypes ‘AA’, ‘Aa’, ‘aa’, and potentially ‘NN’ for missing data. The names of the vector should be ‘AA’, ‘Aa’, ‘aa’, and optionally ‘NN’.

Details

The function first calculates the total number of valid genotypes by subtracting any missing data (‘NN’) from the total genotype count. It then computes the frequency of the dominant allele ‘A’ (‘p’) and the recessive allele ‘a’ (‘q’) using the formula:

$$p = \frac{(2 \times \text{AA count}) + \text{Aa count}}{2 \times \text{total count}}$$

$$q = 1 - p$$

Value

A named vector containing the frequencies of the dominant allele ‘p’ and the recessive allele ‘q’.

Note

Ensure that 'genotypeCounts' contains valid counts for all three genotypes ('AA', 'Aa', 'aa') and optionally 'NN' (if missing data is present).

Examples

```
# Example usage:
genotypes <- c(AA = 50, Aa = 30, aa = 20, NN = 10) # 10 missing values
alleleFrequencies <- allele.freq(genotypes)

# If no missing values:
genotypes <- c(AA = 50, Aa = 30, aa = 20)
alleleFrequencies <- allele.freq(genotypes)
```

AnnoRan

*Annotate Genomic Regions with Gene Names***Description**

Match genomic intervals, such as runs of homozygosity, to gene annotations.

Usage

```
AnnoRan(data, CHR, starP, endP, annotation)
```

Arguments

data	A data frame containing the regions to annotate.
CHR	Column name or index in 'data' containing chromosome identifiers.
starP	Column name or index in 'data' containing region start positions.
endP	Column name or index in 'data' containing region end positions.
annotation	Either a data frame of gene annotations or a path to a GFF/GTF file. Data frames should contain 'chromosome_name', 'start_position', 'end_position', and 'external_gene_name'.

Value

A data frame with 'ID', 'Range', and 'Matching_Names'. 'Matching_Names' contains comma-separated gene names fully contained within each region, or "NA" when no annotation overlaps.

Examples

```
regions <- data.frame(chr = c("1", "1"), start = c(1000, 5000), end = c(2000, 6500))
genes <- data.frame(
  chromosome_name = c("1", "1"),
  start_position = c(1200, 5500),
  end_position = c(1800, 6000),
  external_gene_name = c("GeneA", "GeneB")
)
AnnoRan(regions, CHR = "chr", starP = "start", endP = "end", annotation = genes)
```

AnnoSin

Annotate Single Genomic Positions with Gene Names

Description

Match SNP or marker positions to gene annotations.

Usage

```
AnnoSin(data, annotation)
```

Arguments

data A data frame with 'CHR' and 'POSITION' columns.

annotation A data frame with 'chromosome_name', 'start_position', 'end_position', and 'external_gene_name' columns.

Value

The input data with an added 'external_gene_name' column.

Examples

```
positions <- data.frame(CHR = c("1", "2"), POSITION = c(1500, 6000))
genes <- data.frame(
  chromosome_name = c("1", "2"),
  start_position = c(1000, 5000),
  end_position = c(2000, 6500),
  external_gene_name = c("GeneA", "GeneB")
)
AnnoSin(positions, genes)
```

calc_r2

calc_r2: Calculate Linkage Disequilibrium (R^2) Between Two Variants

Description

This function calculates the linkage disequilibrium (LD) measure, R^2 , between two genetic variants based on their genotypes. The function uses the allele frequencies of the two variants and their haplotype frequencies to compute R^2 , which quantifies the non-random association of alleles at two loci.

Usage

```
calc_r2(row1, row2)
```

Arguments

- row1

A vector representing the first variant, where the first 9 columns contain meta-data and the genotype data starts from the 10th column onward.
- row2

A vector representing the second variant, structured similarly to ‘row1’, where the genotype data starts from the 10th column.

Details

The function extracts the genotypes (‘GT’) for both variants using the helper function ‘get.field’, which parses the genotype data from the variant vectors. It then calculates the allele frequencies ‘pA’ and ‘pB’ for both variants using the ‘allele.freq’ function. The haplotype frequencies are determined using the ‘get.haplotypes’ function, and from this, the R^2 value is calculated using the formula:

$$D = p_{AB} - (p_A \times p_B)$$
$$R^2 = \frac{D^2}{p_A \times (1 - p_A) \times p_B \times (1 - p_B)}$$

Value

A numeric value representing the R^2 value between the two variants, which measures the degree of linkage disequilibrium between them.

Note

The ‘get.field’, ‘count.genotypes’, ‘allele.freq’, and ‘get.haplotypes’ functions must be defined or available in the user’s environment for this function to work properly.

Examples

```
# Example usage:
variant1 <- c("chr1", "12345", ".", "A", "T", ".", "PASS", ".", "GT", "0/0", "0/1", "1/1")
variant2 <- c("chr1", "67890", ".", "G", "C", ".", "PASS", ".", "GT", "0/0", "0/1", "1/1")
r2_value <- calc_r2(variant1, variant2)
```

count.genotypes	<i>count.genotypes: Count Genotypes from Genotype Data</i>
-----------------	--

Description

This function counts the occurrences of specific genotype patterns (homozygous reference, heterozygous, homozygous alternate, and missing data) from a vector of genotype strings.

Usage

```
count.genotypes(genotypes)
```

Arguments

- genotypes

A character vector containing genotype data in the form of “0|0”, “0/1”, “1|1”, etc. The genotypes can be phased (‘|’) or unphased (‘/’).

Details

The function first removes any phase ('|') or unphase ('/') symbols in the genotypes, normalizing the genotypes to simple two-character strings ("00", "01", "10", "11", or ".."). It then counts the occurrences of each pattern. The heterozygous counts for "01" and "10" are combined into a single count for 'Aa'.

Value

A named numeric vector containing the counts of the genotypes 'AA' (homozygous reference), 'Aa' (heterozygous), 'aa' (homozygous alternate), and 'NN' (missing or unknown).

Note

The function returns a vector with counts for 'AA', 'Aa', 'aa', and 'NN', even if some genotypes are not present in the input.

Examples

```
# Example usage:
genotypes <- c("0|0", "0/1", "1|0", "1|1", "..")
genotype_counts <- count.genotypes(genotypes)

# Output: named vector with counts for AA, Aa, aa, and NN
```

derivedCount	<i>derivedCount: Calculate Derived Allele Count from Genotype Data</i>
--------------	--

Description

This function calculates the derived allele count for a given genetic variant based on its genotype data. The derived allele count is computed as twice the number of homozygous alternate genotypes ('aa') plus the number of heterozygous genotypes ('Aa').

Usage

```
derivedCount(row)
```

Arguments

row	A vector representing a genetic variant, where the first 9 columns contain meta-data, and the genotype data starts from the 10th column onward. The 9th column should contain the format string for genotype fields (e.g., "GT" for genotype).
-----	--

Details

The function extracts the genotype data ('GT') from the 'row' vector using the 'get.field' function. It then uses the 'count.genotypes' function to count the occurrences of 'AA', 'Aa', 'aa', and missing genotypes ('NN'). The derived allele count is computed as:

$$\text{Derived Count} = (2 \times \text{aa count}) + \text{Aa count}$$

Value

A numeric value representing the derived allele count for the variant.

Note

The ‘get.field’ and ‘count.genotypes’ functions must be defined or available in the user’s environment for this function to work.

Examples

```
# Example usage:
variant <- c("chr1", "12345", ".", "A", "T", ".", "PASS", ".", "GT", "0/0", "0/1", "1/1")
derived_allele_count <- derivedCount(variant)
```

dnuc	<i>dnuc: Calculate Nucleotide Diversity (D_{xy}) Between Two Populations</i>
------	---

Description

This function calculates the nucleotide diversity D_{xy} between two populations using allele frequencies from two variant call format (VCF) datasets. The measure quantifies the average number of nucleotide substitutions per site between two populations.

Usage

```
dnuc(vcf1, vcf2, perBP = TRUE)
```

Arguments

vcf1	A data frame representing the VCF data for the first population, where the first 9 columns contain metadata, and the genotype data starts from the 10th column.
vcf2	A data frame representing the VCF data for the second population, structured similarly to ‘vcf1’.
perBP	A logical value indicating whether to normalize D_{xy} per base pair (bp). If ‘TRUE’, the function divides the D_{xy} value by the total base pairs between the start and end positions in ‘vcf1’. Default is ‘TRUE’.

Details

The function works by first extracting the genotype data (‘GT’) from the VCF files using the ‘get.field’ function. It then calculates the allele frequencies (‘p’) for each variant in both populations using ‘allele.freq’. The D_{xy} measure is computed using the formula:

$$D_{xy} = \sum p_1(1 - p_2) + p_2(1 - p_1)$$

Where p_1 and p_2 are the allele frequencies in population 1 and population 2, respectively. If ‘perBP’ is ‘TRUE’, the function calculates the total number of base pairs covered by the variants and returns D_{xy} normalized by this number.

Value

A numeric value representing the D_{xy} value. If 'perBP' is 'TRUE', the value is normalized per base pair, otherwise it returns the raw D_{xy} value.

Note

The 'get.field' and 'allele.freq' functions must be defined or available in the user's environment for this function to work.

Examples

```
# Example usage:
## Not run:
vcf1 <- read.vcf("population1.vcf")
vcf2 <- read.vcf("population2.vcf")
dxy_value <- dnuc(vcf1, vcf2, perBP = TRUE)

## End(Not run)
```

expected.het

expected.het: Calculate Expected Heterozygosity from Genotype Data

Description

This function calculates the expected heterozygosity (Hexp) for a set of genotypes at a bi-allelic locus. Expected heterozygosity is a measure of genetic diversity and is based on allele frequencies.

Usage

```
expected.het(genotypes)
```

Arguments

genotypes A character vector containing genotype data, such as "0|0", "0/1", "1|1", etc. The genotypes can be phased ('|') or unphased ('/').

Details

The function first counts the occurrences of different genotypes using the 'count.genotypes' function. It then calculates the total number of valid (non-missing) genotypes 'n' and the allele frequencies 'p' and 'q' using the 'allele.freq' function. The expected heterozygosity is computed as:

$$H_{exp} = 2 \times p \times q$$

Where 'p' is the frequency of the dominant allele, and 'q = 1 - p' is the frequency of the recessive allele.

Value

A numeric vector containing three elements: the allele frequency 'p' of the dominant allele, the number of non-missing genotypes 'n', and the expected heterozygosity 'Hexp'.

Note

The ‘count.genotypes’ and ‘allele.freq’ functions must be available in the user’s environment.

Examples

```
# Example usage:
genotypes <- c("0|0", "0/1", "1|1", ". /.")
result <- expected.het(genotypes)

# Output: [1] p (allele frequency), n (number of genotypes), Hexp (expected heterozygosity)
```

fixed.poly

fixed.poly: Identify Fixed and Polymorphic Sites Between Two Populations

Description

This function identifies whether a genetic site is fixed or polymorphic between two populations based on allele frequencies calculated from their genotype data.

Usage

```
fixed.poly(vcf1, vcf2)
```

Arguments

vcf1	A data frame representing the VCF data for the first population, where the first 9 columns contain metadata and the genotype data starts from the 10th column.
vcf2	A data frame representing the VCF data for the second population, structured similarly to ‘vcf1’.

Details

The function first extracts the genotype data (‘GT’) from both VCF files using the ‘get.field’ function. It then calculates allele frequencies (‘p’) for each site in both populations using the ‘allele.freq’ and ‘count.genotypes’ functions. A site is considered "Fixed" if the absolute difference in allele frequencies between the two populations is exactly 1, and "Polymorphic" otherwise.

Value

A character vector indicating whether each site is "Fixed" or "Polymorphic" between the two populations.

Note

The ‘get.field’, ‘allele.freq’, and ‘count.genotypes’ functions must be available in the user’s environment for this function to work.

Examples

```
# Example usage:
## Not run:
vcf1 <- read.vcf("population1.vcf")
vcf2 <- read.vcf("population2.vcf")
result <- fixed.poly(vcf1, vcf2)

## End(Not run)

# Output: A vector indicating "Fixed" or "Polymorphic" for each site
```

get.field

get.field: Extract Specific Field from Genotype Data

Description

This function extracts a specific field (such as 'GT', 'DP', etc.) from the genotype information provided in the VCF format for each sample.

Usage

```
get.field(samples, format, fieldName)
```

Arguments

samples	A character vector containing the genotype data for each sample. Each element in the vector typically contains multiple fields separated by colons (e.g., "GT:DP:AD").
format	A character string specifying the format field in the VCF file, which describes the fields present in the genotype data (e.g., "GT:DP:AD").
fieldName	A character string specifying the name of the field to extract (e.g., "GT" for genotype, "DP" for read depth).

Details

The function works by first splitting the 'samples' vector and the 'format' string based on colons (':'). It identifies the position of 'fieldName' within the format and extracts the corresponding values from the 'samples' vector. If 'fieldName' is not found in the format, the function throws an error.

Value

A character vector containing the values of the specified field for each sample.

Note

This function assumes that all samples follow the same format structure.

Examples

```
# Example usage:
samples <- c("0/1:35:99", "1/1:20:85")
format <- "GT:DP:GQ"
field_values <- get.field(samples, format, "GT")

# Output: A vector with extracted genotype values: "0/1", "1/1"
```

get.haplotypes	<i>get.haplotypes: Generate Haplotype Combinations from Genotype Data</i>
----------------	---

Description

This function generates haplotypes by combining the alleles from two sets of genotype data. It concatenates the alleles from the two genotypes to produce haplotype pairs for each sample.

Usage

```
get.haplotypes(genotypes1, genotypes2)
```

Arguments

genotypes1	A character vector containing the genotype data for the first set of samples. The genotypes can be phased (' ') or unphased ('/').
genotypes2	A character vector containing the genotype data for the second set of samples, structured similarly to 'genotypes1'.

Details

The function first removes the phase markers ('|') from the genotype strings, then splits the combined alleles from both genotype sets into individual alleles. It finally concatenates the alleles from 'genotypes1' and 'genotypes2' to create haplotypes for each sample.

Value

A character vector of concatenated haplotypes, where each haplotype is formed by combining alleles from 'genotypes1' and 'genotypes2'.

Note

The input genotype vectors must have the same length, and corresponding entries should represent the same samples.

Examples

```
# Example usage:
genotypes1 <- c("0|1", "1|0", "0|0")
genotypes2 <- c("1|0", "0|1", "1|1")
haplotypes <- get.haplotypes(genotypes1, genotypes2)

# Output: A vector of haplotypes: c("01", "10", "00", "10", "01", "11")
```

Hetp

*Calculate Pooled Heterozygosity by Genomic Window***Description**

Calculate pooled heterozygosity (Hp) from SNP reference-allele counts in fixed-width windows across a VCF file.

Usage

```
Hetp(vcf_file, window_size = 150000)
```

Arguments

`vcf_file` Path to a VCF file.

`window_size` Window size in base pairs. Defaults to 150,000.

Value

A data frame with chromosome, genomic range, maximum and minimum allele counts, Hp, and standardized ZHp values.

Examples

```
## Not run:
Hetp("path/to/sample.vcf", window_size = 150000)

## End(Not run)
```

maf

*maf: Calculate Minor Allele Frequency (MAF) from VCF Row***Description**

This function calculates the minor allele frequency (MAF) for a given row of variant call format (VCF) data. MAF is the frequency of the least common allele at a locus and is a measure of genetic variation.

Usage

```
maf(vcf.row)
```

Arguments

`vcf.row` A vector representing a single row of VCF data, where the first 9 columns contain metadata and genotype information starts from the 10th column.

Details

The function extracts genotype data from the VCF row using the 'get.field' function. It counts the genotypes with 'count.genotypes', calculates allele frequencies using 'allele.freq', and returns the minimum frequency as the MAF.

Value

A numeric value representing the minor allele frequency (MAF) for the specified VCF row.

Note

The 'get.field', 'count.genotypes', and 'allele.freq' functions must be available in the user's environment for this function to work correctly.

Examples

```
# Example usage:
vcf_row <- c("chr1", "123456", ".", "A", "G", ".", "PASS", ".", "GT:DP", "0/1", "1/1", "0/0")
maf_value <- maf(vcf_row)

# Output: Numeric value representing the minor allele frequency
```

Mbed

Create Fixed-Width BED Windows with bedtools

Description

Create genomic windows from a FASTA index or genome file by calling 'bedtools makewindows'.

Usage

```
Mbed(Ffile, Window, output = "test.bed", bedtools = "bedtools")
```

Arguments

Ffile	Path to a genome file accepted by 'bedtools makewindows -g'.
Window	Window size in base pairs.
output	Output BED file path. Defaults to 'test.bed'.
bedtools	Path or command name for bedtools.

Value

Invisibly returns 'output' after bedtools completes.

Examples

```
## Not run:
Mbed("genome.fai", Window = 40000, output = "windows.bed")

## End(Not run)
```

pi.diversity

*pi.diversity: Calculate Nucleotide Diversity (Pi) from VCF Data***Description**

This function calculates the nucleotide diversity (Pi) for a given VCF data frame, which quantifies genetic variation at a particular site or across multiple sites in a population.

Usage

```
pi.diversity(vcf, perBP = TRUE)
```

Arguments

vcf	A data frame representing the VCF data, where the first 9 columns contain meta-data and the genotype data starts from the 10th column.
perBP	A logical value indicating whether to normalize the nucleotide diversity by the number of base pairs. If 'TRUE', the result is divided by the total number of base pairs; if 'FALSE', the raw nucleotide diversity is returned.

Details

The function computes the derived count for each site using the 'derivedCount' function. It calculates the total number of alleles ('C') and uses the formula for nucleotide diversity:

$$\pi = \frac{1}{N(N-1)} \sum_{i=1}^S J_i(C - J_i)$$

where J is the derived count, and N is the total number of alleles. If 'perBP' is set to 'TRUE', the function normalizes the result by the total number of base pairs across all chromosomes.

Value

A numeric value representing the nucleotide diversity (Pi). If 'perBP' is 'TRUE', the result is normalized by the number of base pairs.

Note

The 'derivedCount', 'count.genotypes', and other necessary functions must be available in the user's environment for this function to work properly.

Examples

```
# Example usage:
## Not run:
vcf_data <- read.vcf("sample.vcf")
nucleotide_diversity <- pi.diversity(vcf_data, perBP = TRUE)

## End(Not run)

# Output: A numeric value representing nucleotide diversity normalized by base pairs
```

plQC

Run PLINK Quality Control and Plot PCA

Description

Filter genotype data with PLINK and plot the first two principal components.

Usage

```
plQC(  
  file,  
  maf = 0.05,  
  gen = 0.1,  
  mind = 0.1,  
  hwe = 1e-06,  
  chrset = 22,  
  plink_path = NULL  
)
```

Arguments

file	Input PLINK file prefix, without ‘.ped’ or ‘.bed’.
maf	Minor allele frequency threshold.
gen	Variant missingness threshold passed to PLINK ‘–geno’.
mind	Individual missingness threshold passed to PLINK ‘–mind’.
hwe	Hardy-Weinberg equilibrium p-value threshold.
chrset	Number of chromosomes passed to PLINK ‘–chr-set’.
plink_path	Optional path to a PLINK executable. When omitted, the function looks for a package/local development copy and then for ‘plink’ on the system path.

Value

A ‘ggplot’ object. PLINK output files are written to the current working directory.

Examples

```
## Not run:  
plQC(file = "data/my_genotypes", maf = 0.05, chrset = 30)  
  
## End(Not run)
```

read.vcf

*read.vcf: Read and Clean VCF Files***Description**

This function reads a Variant Call Format (VCF) file, removes specific comment lines, and returns the data as a data frame.

Usage

```
read.vcf(file, special.char = "##", ...)
```

Arguments

<code>file</code>	A character string representing the path to the VCF file to be read.
<code>special.char</code>	A character string that indicates the type of comment lines to be removed from the VCF file. The default is <code>"##"</code> which is typical for VCF files.
<code>...</code>	Additional arguments to be passed to the <code>'read.table'</code> function for reading the data.

Details

The function reads all lines from the specified VCF file, removes lines starting with the specified comment character (defaulting to `'##'`), and replaces the header line containing `'#CHROM'` with `'CHROM'`. The cleaned lines are then converted into a data frame using `'read.table'`.

Value

A data frame containing the cleaned VCF data, with appropriate column names for the genomic data.

Note

The input file must be a valid VCF file. The function assumes that the format of the file is correct and consistent with VCF standards.

Examples

```
# Example usage:
## Not run:
vcf_data <- read.vcf("path/to/sample.vcf")

## End(Not run)

# Output: A data frame representing the cleaned VCF data
```

RunHom*Run PLINK Runs of Homozygosity*

Description

Detect runs of homozygosity (ROH) with PLINK.

Usage

```
RunHom(  
  file,  
  homozygWS = 50,  
  homozygS = 50,  
  homozygWM = 3,  
  homozygKB = 100,  
  homozygDN = 1000,  
  chrset = 22,  
  plink_path = NULL  
)
```

Arguments

file	Input PLINK file prefix, without '.ped' or '.bed'.
homozygWS	PLINK '--homozyg-window-snp' value.
homozygS	PLINK '--homozyg-snp' value.
homozygWM	PLINK '--homozyg-window-missing' value.
homozygKB	PLINK '--homozyg-kb' value.
homozygDN	PLINK '--homozyg-density' value.
chrset	Number of chromosomes passed to PLINK '--chr-set'.
plink_path	Optional path to a PLINK executable.

Value

A data frame read from 'roh.hom', when PLINK produces it.

Examples

```
## Not run:  
roh <- RunHom(file = "data/my_genotypes", chrset = 30)  
  
## End(Not run)
```

TajimaD

*Calculate Tajima's D by Genomic Window***Description**

Calculate Tajima's D and supporting diversity statistics in fixed-width windows from a VCF file.

Usage

```
TajimaD(vfile, window_size = 150000, nsample = NULL)
```

Arguments

vfile	Path to a VCF file.
window_size	Window size in base pairs. Defaults to 150,000.
nsample	Optional number of samples. When 'NULL', the sample count is inferred from the VCF genotype columns.

Value

A data frame with one row per genomic window and columns for SNP count, chromosome count, Watterson's theta, nucleotide diversity, variance, and Tajima's D.

Examples

```
## Not run:
TajimaD("path/to/sample.vcf", window_size = 150000)

## End(Not run)
```

variance.d

*variance.d: Calculate Variance of the Genetic Diversity Index (D)***Description**

This function calculates the variance of the genetic diversity index (D) using the number of samples and the observed diversity.

Usage

```
variance.d(n, S)
```

Arguments

n	An integer representing the number of samples.
S	A numeric value representing the observed genetic diversity.

Details

The function uses the following formulas to compute the variance:

$$a1 = \sum_{i=1}^{n-1} \frac{1}{i}$$

$$a2 = \sum_{i=1}^{n-1} \frac{1}{i^2}$$

Then it calculates coefficients $b1$, $b2$, $c1$, and $c2$ based on these sums, and finally computes the variance using:

$$var = e1 \cdot S + e2 \cdot S \cdot (S - 1)$$

where $e1$ and $e2$ are derived from $c1$ and $c2$.

Value

A numeric value representing the variance of the genetic diversity index (D).

Note

This function assumes that the input values are valid and that the calculations adhere to the statistical properties of genetic diversity.

Examples

```
# Example usage:
n_samples <- 10
observed_diversity <- 0.5
diversity_variance <- variance.d(n_samples, observed_diversity)

# Output: A numeric value representing the variance of the genetic diversity index
```

waterson.theta

waterson.theta: Calculate Watterson's Theta from Genetic Data

Description

This function calculates Watterson's Theta θ , a measure of nucleotide diversity, based on the number of segregating sites in a given dataset. It can be normalized by the number of base pairs.

Usage

```
waterson.theta(data, perBP = TRUE)
```

Arguments

data	A data frame containing genetic data in VCF format, where the first 9 columns contain metadata and genotype data starts from the 10th column.
perBP	A logical value indicating whether to normalize Watterson's Theta by the number of base pairs. If 'TRUE', the result is divided by the total number of base pairs; if 'FALSE', the raw value is returned.

Details

The function calculates the number of segregating sites S_n in the data that have a minor allele frequency greater than zero. It then uses the formula:

$$\theta = \frac{S_n}{\sum_{i=1}^{2N-1} \frac{1}{i}}$$

where N is the number of individuals in the sample. If 'perBP' is set to 'TRUE', the result is normalized by the number of base pairs in the dataset.

Value

A numeric value representing Watterson's Theta. If 'perBP' is 'TRUE', the result is normalized by the number of base pairs.

Note

The 'maf' function must be available in the user's environment for this function to work correctly.

Examples

```
# Example usage:
## Not run:
vcf_data <- read.vcf("path/to/sample.vcf")
watterson_theta <- watterson.theta(vcf_data, perBP = TRUE)

## End(Not run)

# Output: A numeric value representing Watterson's Theta normalized by base pairs
```